Bad-Photo Detector: Codex Implementation Plan

Objective

Implement a focused experiment that ranks Vinted listings by the technical quality of their first image.

The system must answer only:

> "How likely is this first image to contain a meaningful technical or presentation defect?"

Do not evaluate:

- whether the product is valuable;
- whether the price is good;
- resale profitability;
- brand desirability;
- seller quality;
- likely time to sell;
- whether the user should buy the item.

The human user will review commercial value separately.

Main decision

Do not make product-type classification a required stage.

The new detector should score photo defects independently of whether the listing contains clothing, perfume, shoes, electronics, games, or another product.

Keep the existing generic CLIP detector and typed CLIP detector only as comparison baselines.

Do not spend time fixing all product-type regex rules unless needed to rerun the old typed baseline.

Working location

First check whether this local directory exists:

```
experiments/current/multimodal_beat/
```

If it exists, add the new experiment there under:

```
experiments/current/multimodal_beat/bad_photo_eval/
```

If it does not exist in the current checkout, create:

```
experiments/current/bad_photo_detection/
```

Do not place new work under `/home/ale/Desktop`.

Generated files must go under the experiment's `data/` directory and remain excluded from Git when large.

Existing code to reuse

Reuse relevant code from:

```
experiments/old/photo_arbitrage/features.py
experiments/old/photo_arbitrage/quality_methods.py
experiments/old/photo_arbitrage/modeling.py
experiments/old/photo_arbitrage/compare_quality_methods.py
```

Reuse only components related to image quality:

- image path resolution;
- brightness;
- contrast;
- saturation;
- sharpness;
- edge density;
- width and height;
- image-review HTML generation;
- optional PyIQA scoring;
- optional FashionCLIP model loading.

Do not include these fields in the technical bad-photo score:

- price;
- brand;
- condition;
- seller reviews;
- seller rating;
- number of listing photos;
- duplicate-photo count;
- missing brand;
- any opportunity or business score.

Number of photos can be retained as metadata but must not affect first-image technical quality.

Phase 1: Recover and lock embedding alignment

The cached files are:

```
cache/img_clip_live19k.npy
cache/img_clip_live19k_mask.npy
```

Expected values:

```
CSV rows:       20,450
Embedding rows: 20,450
Embedding dim:  512
Valid images:   19,173
```

Find the exact CSV whose row order matches the embeddings.

Search the repository and local experiment data for candidate CSV files with 20,450 rows.

Do not assume that a CSV is aligned only because it has the correct row count.

Use available metadata, item IDs, image paths, previous script arguments, shell history or experiment manifests to prove the alignment.

If alignment cannot be established reliably, stop with a clear error and report the candidate files found. Do not run the experiment on guessed alignment.

Once recovered, create:

```
cache/img_clip_live19k_item_ids.npy
cache/img_clip_live19k_manifest.json
```

The manifest must include:

```
{
  "source_csv": "...",
  "source_csv_sha256": "...",
  "row_count": 20450,
  "embedding_shape": [20450, 512],
  "mask_shape": [20450],
  "valid_image_count": 19173,
  "embedding_model": "openai/clip-vit-base-patch32",
  "normalized_embeddings": true,
  "created_at": "...",
  "item_id_column": "..."
}
```

Every scoring script must assert:

```
assert len(df) == len(embeddings)
assert len(df) == len(mask)
assert len(df) == len(item_ids)
assert embeddings.shape[1] == 512
assert df[item_id_column].astype(str).tolist() ==
item_ids.astype(str).tolist()
```

Normalize item IDs before comparing:

- convert to strings;
- strip whitespace;
- remove trailing `.0`;
- treat missing IDs as errors for evaluation rows.

Phase 2: Create a globally deduplicated listing table

The same Vinted listing can occur under multiple searches.

Deduplicate globally by normalized `item_id`.

Do not discard search information. Aggregate it:

```
item_id: 9090394550
search_names: ["gucci", "griffati_uomo_all"]
```

Keep one image embedding and one image path per unique listing.

The deduplicated table should contain at least:

```
item_id
search_names
primary_search
title
image_path
image_available
original_row_indices
```

Use stable and deterministic ordering.

Phase 3: Implement generic CLIP defect margins

Create a prompt configuration file, for example:

```
prompts.py
```

Implement independent prompt groups for:

- blur;
- exposure;
- glare;
- crop/framing;
- low resolution/noise;
- clutter/item visibility;
- tilt.

Each defect must have matched good and bad prompts.

Example structure:

```
DEFECT_PROMPTS = {
    "blur": {
        "good": [
            "a sharply focused photograph",
            "a crisp photograph with clearly visible fine detail",
            "a photograph without motion blur",
        ],
        "bad": [
            "a visibly out of focus photograph",
            "a photograph with strong motion blur",
            "a blurry photograph with lost fine detail",
        ],
    },
}
```

Keep prompts generic. Do not mention clothing, shoes, perfume, phones or other product types.

Embed every text prompt once and cache the normalized text embeddings.

For each image and defect, calculate:

```
bad_similarity = mean(image_embedding @ bad_prompt_embeddings.T)
good_similarity = mean(image_embedding @ good_prompt_embeddings.T)
defect_margin = bad_similarity - good_similarity
```

Do not apply:

- softmax across prompts;
- sigmoid;
- probability calibration;
- type-specific prompt selection.

Store raw values:

```
clip_blur_bad_similarity
clip_blur_good_similarity
clip_blur_margin

clip_exposure_bad_similarity
clip_exposure_good_similarity
clip_exposure_margin

...
```

Define the overall CLIP score as:

```
clip_overall_margin = max(all_defect_margins)
```

Also save:

```
clip_top_defect
clip_second_defect
clip_top_margin
clip_second_margin
```

This makes each result explainable.

Optionally save the mean of the positive margins as a diagnostic score, but do not use it as the primary rank.

Phase 4: Implement a simple technical baseline

Create a first-image-only baseline using cheap image measurements.

Reuse existing photo-arbitrage feature extraction where appropriate.

Required features:

Resolution

```
width
height
minimum_dimension
pixel_count
```

Exposure

```
mean_luminance
luminance_std
```

```
dark_pixel_fraction
bright_pixel_fraction
dynamic_range
```

Suggested definitions:

```
dark_pixel_fraction = mean(luminance < 0.05)
bright_pixel_fraction = mean(luminance > 0.95)
dynamic_range = percentile_95 - percentile_5
```

Blur

```
gradient_sharpness
laplacian_variance
tenengrad
edge_density
```

Calculate sharpness at more than one scale where inexpensive.

Glare proxy

```
high_brightness_low_saturation_fraction
largest_highlight_region_fraction
```

A glare proxy is allowed to be imperfect. Store it independently instead of pretending it is calibrated.

Framing proxies

At minimum add:

```
aspect_ratio
centre_edge_density
border_edge_density
centre_to_border_edge_ratio
```

Do not introduce a large object-detection or segmentation model in this phase.

Technical baseline score

Do not hard-code a supposedly calibrated probability.

Convert each feature into a within-dataset percentile representing worse quality.

Examples:

```
lower sharpness → higher bad percentile
lower resolution → higher bad percentile
higher dark fraction → higher bad percentile
higher bright fraction → higher bad percentile
higher glare proxy → higher bad percentile
```

Create:

```
simple_blur_score
simple_exposure_score
simple_resolution_score
simple_glare_score
simple_framing_score
```

Define:

```
simple_overall_score = max(
    simple_blur_score,
    simple_exposure_score,
    simple_resolution_score,
    simple_glare_score,
    simple_framing_score,
)
```

Store the dominant simple-feature defect.

Do not mix this score with CLIP yet.

Phase 5: Preserve the existing methods as baselines

Where possible, rerun and store:

```
v1_generic_clip_score
v2_typed_clip_score
```

Do not modify their scoring behavior before evaluation. They are frozen baselines.

If the original scripts cannot be rerun safely because of missing alignment information, preserve any retained scores that can be matched by `item_id`.

Clearly mark unavailable baseline scores instead of substituting new behavior under an old method name.

The methods to compare should be:

```
v1_generic_clip
v2_typed_clip
v3_generic_defect_margin
simple_technical
```

Optional methods may be included as secondary diagnostics:

```
PyIQA/MUSIQ
FashionCLIP existing implementation
aesthetic model
```

Do not allow optional model-loading failures to stop the core CLIP-margin and simple-feature experiment.

Phase 6: Produce one scoring table

Write:

```
data/scored_bad_photo_candidates.csv
```

One row per unique `item_id`.

Include:

```
item_id
search_names
title
image_path

v1_generic_clip_score
v2_typed_clip_score

clip_overall_margin
clip_top_defect
all individual CLIP defect margins

simple_overall_score
simple_top_defect
all simple technical features and component scores

pyiqa score if available
fashionclip score if available

source CSV hash
```

```
embedding-cache identifier
scoring version
```

Do not calculate a combined final score before manual evaluation.

Phase 7: Build a small blind evaluation set

The existing manual labels are too few to train or select a final model reliably.

Use them only as possible overlap/reference rows. Do not optimize prompts against them.

Create a new fixed evaluation set.

Use these searches when present:

```
prada
gucci
nike
ps4
griffati_donna_all
griffati_uomo_all
```

For each search, select:

```
8 highest-ranked by v1
8 highest-ranked by v2
8 highest-ranked by v3 generic CLIP margin
8 highest-ranked by simple technical score
8 random rows outside the top-ranked region
```

Maximum before overlap:

```
6 searches × 40 = 240 listings
```

Rules:

1. Deduplicate globally by `item_id`.
2. Keep all selection reasons for internal evaluation.
3. Shuffle gallery order using a fixed random seed.
4. Hide method, score, rank and selection reason from the labeler.
5. Do not show inferred product type.
6. Add approximately 10% repeated images under different blind review IDs to measure labeling consistency.
7. Do not count repeated images in final precision metrics.

Write:

```
data/evaluation/eval_manifest.json
data/evaluation/eval_candidates_private.csv
data/evaluation/blind_label_sheet.csv
data/evaluation/blind_gallery.html
```

The private file contains scores and selection buckets.

The blind file must not contain them.

Phase 8: Label schema

The HTML gallery and CSV should support these labels.

Primary label:

```
technical_quality:
- good
- bad
- uncertain
- not_item_photo
```

Secondary label:

```
hurts_listing_presentation:
- yes
- no
- uncertain
```

Fixability label:

```
fixable_by_retake:
- yes
- no
- uncertain
```

Defect tags must allow multiple values:

```
blur
dark
overexposed
glare
bad_crop
extreme_tilt
low_resolution
noise
```

```
clutter
item_not_clear
other
```

The main evaluation positive is:

```
technical_quality == bad
```

Also evaluate the stricter target:

```
technical_quality == bad
and hurts_listing_presentation == yes
and fixable_by_retake == yes
```

Keep `uncertain` separate. Do not force uncertain rows into good or bad.

Phase 9: Evaluation script

Create an evaluation script that merges the completed blind labels back with the private score file using the blind review ID.

Report for each method:

```
precision@3
precision@5
precision@8
```

Calculate results:

- for each search;
- macro-averaged across searches;
- pooled across all searches;
- by defect tag where sample size allows;
- for the strict fixable-and-harmful target;
- lift over the random bucket.

Primary metric:

```
macro precision@8 across searches
```

Also report:

```
random bad-photo prevalence
uncertain-label rate
```

```
duplicate-review agreement
number of unique items surfaced
```

Do not report dataset-wide recall from this enriched sample.

Write:

```
data/evaluation/metrics_summary.json
data/evaluation/metrics_by_method.csv
data/evaluation/metrics_by_search.csv
data/evaluation/metrics_by_defect.csv
data/evaluation/false_positives.csv
data/evaluation/false_negatives_from_random.csv
data/evaluation/results.md
```

Decision rule

The new generic CLIP-margin method is worth retaining if:

```
macro precision@8 >= 0.50
and
lift over random >= 2.0
and
precision@8 >= 0.375 in at least 4 searches
```

It should also beat the better legacy CLIP method by at least 0.10 absolute macro precision@8, or provide similarly strong precision with much clearer defect explanations.

If the simple technical baseline beats CLIP, prioritize the simple baseline.

If neither reaches 0.50 macro precision@8:

- stop prompt tuning;
- do not create more product-type rules;
- use the new evaluation labels to train a small model later.

Do not train that model as part of this task unless there are at least approximately:

```
50 clear bad labels
50 clear good labels
```

The current tiny label set is insufficient for choosing a trustworthy classifier.

Phase 10: Optional supervised experiment only after labeling

Do not run this during the initial implementation.

Prepare the code so that a future model can use:

```
simple technical features
individual CLIP defect margins
optional frozen 512-dimensional CLIP embedding
```

Use logistic regression first.

Evaluation must group duplicate listings and preferably use search-aware or grouped cross-validation.

Do not use product price, brand, seller data or resale-value signals.

Tests

Add tests covering:

Alignment

- mismatched row count fails;
- mismatched item-ID order fails;
- correct aligned sidecar passes;
- missing item IDs fail clearly.

Deduplication

- one `item_id` appearing in two searches produces one scored row;
- both search names are retained;
- duplicate rows cannot appear twice in the blind gallery.

CLIP scoring

- prompt count does not change the scale through softmax competition;
- no softmax or sigmoid is used;
- a defect margin equals mean bad similarity minus mean good similarity;
- overall score equals the maximum defect margin;
- text embeddings are normalized and cached.

Evaluation set

- deterministic output for the same random seed;
- method metadata is absent from the blind label sheet;
- random rows do not come from the selected top-ranking region;
- hidden repeat rows are excluded from the unique-item metric denominator.

Metrics

- precision@K is correct;
- macro and pooled precision are calculated separately;
- uncertain labels are excluded, not converted;
- strict fixable target is calculated correctly.

Documentation

Create a README containing:

```
goal
non-goals
data alignment requirements
method definitions
commands
output files
label instructions
metric definitions
decision rule
known limitations
```

Explicitly document:

- scores are ranking scores, not calibrated probabilities;
- product type is not required;
- no business-value decision is performed;
- only the first image is evaluated;
- manual labels must remain blind to the model scores.

Suggested command interface

Use commands similar to:

```
python score_bad_photos.py
  --input-csv <aligned_csv>
  --embeddings cache/img_clip_live19k.npy
  --mask cache/img_clip_live19k_mask.npy
  --item-ids cache/img_clip_live19k_item_ids.npy
  --out data/scored_bad_photo_candidates.csv
```

```
python build_bad_photo_eval.py
  --scores data/scored_bad_photo_candidates.csv
  --per-method-search 8
  --random-per-search 8
  --repeat-fraction 0.10
  --seed 24072026
  --out-dir data/evaluation
```

```
python evaluate_bad_photo_eval.py
  --private-candidates data/evaluation/eval_candidates_private.csv
  --labels data/evaluation/blind_label_sheet.csv
  --out-dir data/evaluation
```

Add a smoke-test option:

```
--max-items 100
```

Implementation order

Perform work in this order:

1. Inspect repository structure and existing experiment files.
2. Recover and verify embedding-to-CSV alignment.
3. Add item-ID sidecar and manifest.
4. Implement global item deduplication.
5. Implement generic CLIP defect margins.
6. Implement simple first-image technical features.
7. Generate the combined scoring table.
8. Build the blind evaluation set and gallery.
9. Add evaluation metrics.
10. Add tests and documentation.
11. Run a small smoke test.
12. Run the full scoring pass only after all alignment assertions pass.

Final Codex report

At completion, report:

1. files created and modified;
2. exact source CSV used;
3. proof that embedding alignment passed;
4. unique item count after deduplication;
5. valid image count;
6. which optional models were available;
7. path to the blind gallery;
8. commands for labeling and evaluation;
9. tests run and their results;
10. any remaining blocker.

Do not claim that a method works until the blind labels have been completed and evaluated.